

Universidad Antonio Nariño

Ingeniería de Sistemas

**Protocolo distribuido de control y
reconfiguración topológica con continuidad
lógica para redes multsalto sobre Wi-Fi
convencional**

Protocolo de Control Topológico (PCT)

Juan Carlos Clavijo Triviño
Brandon Stiven Ganzo Murcia

Bogotá D.C.
2026

Índice general

1. Introducción	1
1.1. Planteamiento del problema	1
1.2. Justificación	1
1.3. Objetivo general	2
1.4. Objetivos específicos	2
1.5. Alcance y limitaciones	2
1.5.1. Alcance	2
1.5.2. Limitaciones	2
1.6. Estructura del documento	3
2. Marco teórico y estado del arte	4
2.1. Redes <i>ad hoc</i> y multisalto	4
2.2. Redes mesh en dispositivos móviles	4
2.3. Wi-Fi Direct y SoftAP	4
2.4. DNS-SD y descubrimiento de servicios	5
2.5. Modelo OSI como referencia conceptual	5
2.6. Estado del arte y posicionamiento de PCT	5
2.7. Síntesis del capítulo	5
3. Marco referencial y antecedentes	6
3.1. Antecedentes del proyecto	6
3.2. Prototipo EXP-01 (proto-exp01-gostat)	6
3.3. Evolución hacia lib-pct-core	6
3.4. Requisitos funcionales	7
3.5. Requisitos no funcionales	7
3.6. Restricciones de plataforma Android	8
3.7. Perfiles de hardware	8
3.8. Síntesis del capítulo	8
4. Metodología	9
4.1. Tipo de investigación	9
4.2. Enfoque incremental	9
4.3. Fases de implementación	9
4.4. Matriz de validación experimental	10
4.4.1. Hipótesis	10
4.4.2. Experimentos	10
4.5. Entorno de laboratorio	10
4.6. Técnicas de recolección de datos	10

4.7. Síntesis del capítulo	11
5. Análisis y diseño del sistema	12
5.1. Arquitectura general	12
5.2. Modo de conexión: GO homogéneo + STA legacy	12
5.2.1. Principio rector	12
5.2.2. Dos interfaces lógicas en un nodo BRIDGE	12
5.2.3. Secuencia bootstrap miembro	13
5.3. Capa de enlace (L2)	13
5.3.1. Dos canales TCP por vecino	13
5.3.2. NeighborRegistry	13
5.3.3. Secuencia L2	13
5.4. Capa de red (L3)	14
5.4.1. RoutingEntry	14
5.4.2. Fusión TOPO_UPDATE	14
5.4.3. ForwardWorker	14
5.5. Mensajes dirigidos y difusión	14
5.5.1. Unicast	14
5.5.2. Difusión acotada	14
5.6. Desconexiones en BRIDGE	14
5.7. Unión de árboles independientes	15
5.8. Servicios DNS-SD	15
5.9. Máquinas de estado	15
5.10. Síntesis del capítulo	15
6. Desarrollo e implementación	16
6.1. Stack tecnológico	16
6.2. Estructura de lib-pct-core	16
6.3. Capa física y bootstrap (L1)	16
6.3.1. P2pChannelHolder	16
6.3.2. GoRepository	17
6.3.3. LegacyStaRepository	17
6.3.4. DnsSdRepository	17
6.3.5. Bootstrap automático (PctNodeImpl)	17
6.4. Capa de enlace (L2)	17
6.4.1. PctFrameCodec	17
6.4.2. PctLinkSocket	17
6.4.3. LinkOrchestrator	17
6.4.4. Identidad UUID (PctNid)	18
6.5. Capa de red (L3)	18
6.5.1. RouteOrchestrator	18
6.5.2. ForwardWorker y sendUser	18
6.6. Aplicación demo PCT Mesh	18
6.7. Convenciones de empaquetado UAN	18
6.8. Síntesis del capítulo	18

7. Pruebas, validación y resultados	19
7.1. Plan de pruebas	19
7.2. EXP-01: Interconexión dos GO	19
7.3. EXP-03: Cadena tres nodos	20
7.4. EXP-04 y EXP-05: Unión tardía y fallo BRIDGE	20
7.5. Pruebas unitarias	20
7.6. Análisis de logs	20
7.7. Discusión	20
7.8. Síntesis del capítulo	20
8. Conclusiones y trabajo futuro	21
8.1. Conclusiones	21
8.2. Cumplimiento de objetivos	21
8.3. Limitaciones del trabajo	21
8.4. Trabajo futuro	22
8.5. Cierre	22

Capítulo 1

Introducción

1.1. Planteamiento del problema

Las redes de datos en entornos sin infraestructura fija —desastres naturales, operaciones de campo, comunidades rurales o escenarios tácticos— requieren conectividad entre dispositivos móviles sin depender de torres celulares ni puntos de acceso cableados. Las redes *ad hoc* y *mesh* permiten que cada nodo actúe como repetidor, formando topologías multisalto donde un mensaje puede atravesar varios dispositivos intermedios antes de llegar a su destino lógico.

En smartphones comerciales con Android, las opciones tradicionales presentan limitaciones severas: *Wi-Fi Direct* fue concebido para enlaces punto a punto entre aplicaciones, no para construir árboles de decenas de nodos; modificar la capa MAC o inyectar tramas 802.11 exige *root*; y el *NAT* del kernel no ofrece enrutamiento lógico por identidad de aplicación. Un protocolo de capa de aplicación que emule funciones de red sobre Wi-Fi convencional, sin privilegios elevados, constituye una línea de investigación pertinente para ingeniería de sistemas.

Este proyecto de grado propone el **Protocolo de Control Topológico (PCT)**: un protocolo distribuido de control y reconfiguración topológica con continuidad lógica para redes multisalto sobre Wi-Fi convencional en dispositivos Android 12+ (API 31).

1.2. Justificación

La justificación del trabajo reposa en cuatro ejes:

1. **Accesibilidad hardware.** Aprovechar smartphones comerciales con Wi-Fi Direct (mandatorio en CDD Android) evita hardware especializado.
2. **Modelo homogéneo.** Exigir que todo nodo sea Group Owner (GO) de su propio grupo simplifica el arranque y la re-convergencia tras fallos parciales.
3. **Identidad lógica estable.** Enrutar por UUID (`node_id`) desacopla la topología lógica de las direcciones IPv4 locales, que cambian en cada subred SoftAP.
4. **Continuidad de sesión.** Mantener `session_epoch` ante reconexiones reduce la pérdida de contexto conversacional en aplicaciones de mensajería mesh.

1.3. Objetivo general

Diseñar, implementar y validar un protocolo distribuido de control topológico (PCT) que permita formar y operar redes multisalto sobre Wi-Fi convencional en Android, con enrutamiento lógico por identidad, reconfiguración sin reinicio global y continuidad de sesión ante cambios de ruta.

1.4. Objetivos específicos

1. Definir la arquitectura del protocolo PCT, incluyendo planos de descubrimiento, control y datos, con trazabilidad a requisitos funcionales y no funcionales.
2. Especificar el modelo de enlace basado en GO homogéneo y asociación STA legacy al SoftAP del padre, prohibiendo `WifiP2pManager.connect()` para topología.
3. Diseñar la capa de enlace (L2) con dos canales TCP por vecino (control :8765 y datos :8766) y registro formal de vecinos (`NeighborRegistry`).
4. Diseñar la capa de red (L3) con tabla de rutas local, propagación `TOPO_UPDATE` y reenvío multisalto de tráfico usuario.
5. Implementar la biblioteca `lib-pct-core` (`co.uan.pct:core`) y una aplicación demo que evidencie bootstrap, topología y mensajería.
6. Validar experimentalmente las hipótesis H1–H5 mediante la matriz EXP-01 a EXP-06 en laboratorio con dispositivos clase α .

1.5. Alcance y limitaciones

1.5.1. Alcance

- Plataforma Android 12+ (`minSdk 31`), Kotlin, biblioteca Maven publicable.
- Topología física en árbol: un enlace STA upstream por nodo en el perfil mínimo.
- Profundidad máxima de siete saltos lógicos (`HOP_LIMIT_MAX`).
- Descubrimiento bootstrap vía DNS-SD P2P (`_pct-ctrl`, `_pct-seek`).
- Codificación wire binaria big-endian (v1).

1.5.2. Limitaciones

- Sin cifrado extremo a extremo (RF-17, fuera de alcance v1).
- Sin QoS diferenciado (RF-18).
- PSK del GO en TXT DNS-SD solo para laboratorio controlado.
- Perfil hardware mínimo: clase α (dual GO+STA concurrente verificado).
- Sin modificación de driver Wi-Fi ni acceso MAC/PHY.

1.6. Estructura del documento

El Capítulo 2 presenta fundamentos de redes mesh, Wi-Fi Direct y el modelo OSI emulado. El Capítulo 3 expone antecedentes, requisitos y restricciones Android. El Capítulo 4 describe la metodología incremental y la matriz experimental. El Capítulo 5 detalla el análisis y diseño del sistema PCT. El Capítulo 6 documenta la implementación en `lib-pct-core`. El Capítulo 7 reporta pruebas y resultados. El Capítulo 8 cierra con conclusiones y trabajo futuro.

Capítulo 2

Marco teórico y estado del arte

2.1. Redes *ad hoc* y multisalto

Una red *ad hoc* móvil (MANET) es una colección de nodos inalámbricos que se comunican sin infraestructura preexistente [5]. Cada nodo puede actuar como host y como router. En topologías multisalto, un paquete atraviesa nodos intermedios antes de alcanzar el destino final.

Los protocolos de enrutamiento MANET se clasifican en proactivos (mantienen tablas actualizadas continuamente, p. ej. OLSR) y reactivos (descubren rutas bajo demanda, p. ej. AODV, DSDV) [4]. En entornos con recursos limitados de batería y CPU —como smartphones— el equilibrio entre overhead de control y latencia de convergencia es crítico.

2.2. Redes mesh en dispositivos móviles

Soluciones comerciales y de investigación han explorado mesh sobre Android mediante Wi-Fi Direct, Bluetooth o tunelización. Proyectos como Serval Mesh y Commotion Wireless demostraron viabilidad en escenarios comunitarios, pero suelen requerir *root*, firmware modificado o arquitecturas distintas al perfil homogéneo GO+STA que propone PCT.

PCT se distingue por operar **exclusivamente en espacio de usuario**, sin `connect()` P2P para topología, y por separar explícitamente identidad lógica (UUID) de dirección IPv4 local.

2.3. Wi-Fi Direct y SoftAP

Wi-Fi Direct (Wi-Fi P2P) permite que dispositivos negocien un grupo con un Group Owner (GO) que actúa como SoftAP [6]. Android expone la API `WifiP2pManager` con operaciones `createGroup()`, `removeGroup()` y `requestGroupInfo()`.

La asociación STA legacy (`WifiNetworkSpecifier` desde API 29) permite que un dispositivo se conecte al SSID/PSK de otro GO como estación infraestructura, habilitando el patrón “doble interfaz lógica”: GO propio para hijos downstream y STA upstream hacia el padre.

2.4. DNS-SD y descubrimiento de servicios

DNS-Based Service Discovery (DNS-SD) sobre mDNS permite anunciar y descubrir servicios en redes locales [1]. Android Wi-Fi P2P integra DNS-SD mediante `WifiP2pDnsSdServiceInfo` y `addLocalService()`. PCT utiliza registros TXT compactos para transportar `node_id`, rol, SSID y PSK del GO en la fase bootstrap (capa L1).

2.5. Modelo OSI como referencia conceptual

PCT no implementa una pila OSI completa; utiliza el modelo como **mapa de responsabilidades**:

Tabla 2.1: Correspondencia capas OSI – PCT

Capa OSI	Capa PCT	Transporte
Física (L1)	Radio / bootstrap	802.11, DNS-SD
Enlace (L2)	Vecindad	TCP :8765, :8766
Red (L3)	Reenvío lógico	DATA en :8766
Aplicación	Mensajería usuario	Envelope <code>type=user</code>

La regla normativa es que la capa de red **nunca** enruta por IP de destino final; la IPv4 (`next_hop_local_ip`) es un dato de enlace para alcanzar al vecino inmediato.

2.6. Estado del arte y posicionamiento de PCT

Protocolos como B.A.T.M.A.N. [3] operan en capa 2 con visión global de vecinos por broadcast periódico. Babel [2] es un protocolo de enrutamiento vector-distancia para IPv6. Ambos asumen control sobre la interfaz de red a bajo nivel.

PCT adopta un enfoque de **capa de aplicación**: construye topología y enrutamiento sobre TCP confiable vecino-a-vecino, con identidad UUID y continuidad lógica (`epoch`, `session_epoch`). Esta decisión sacrifica integración con el stack IP del sistema operativo a cambio de despliegue sin privilegios en Android comercial.

2.7. Síntesis del capítulo

El marco teórico establece que las MANET multisalto en móviles requieren equilibrar overhead, identidad estable y restricciones de plataforma. Wi-Fi Direct GO + STA legacy + DNS-SD constituyen la base física de PCT; el modelo OSI emulado organiza las responsabilidades L1–L3 que se diseñan e implementan en capítulos posteriores.

Capítulo 3

Marco referencial y antecedentes

3.1. Antecedentes del proyecto

El presente trabajo de grado se desarrolla en la Universidad Antonio Nariño (UAN) como continuación del anteproyecto titulado *Protocolo distribuido de control y reconfiguración topológica con continuidad lógica para redes multisalto sobre Wi-Fi convencional*. El anteproyecto estableció el planteamiento, objetivos preliminares y alcance; este documento profundiza en diseño formal, implementación y validación experimental.

Los autores —Juan Carlos Clavijo Triviño y Brandon Stiven Ganzo Murcia— dividieron responsabilidades según la arquitectura del sistema: conectividad Wi-Fi (GO, DNS-SD, STA, plano TCP) y enrutamiento/topología (tablas, heartbeats, reenvío).

3.2. Prototipo EXP-01 (proto-exp01-gostat)

Como antecedente de implementación se desarrolló el prototipo experimental EXP-01, documentado en `docs/prototype/proto-exp01-gostat/`. Este prototipo demostró en laboratorio:

- Formación de grupo P2P GO en dos dispositivos Android independientes.
- Descubrimiento DNS-SD del servicio `_pct-ctrl` con parseo de TXT.
- Asociación STA legacy del hijo al SoftAP del padre.
- Coexistencia GO propio + STA upstream (hipótesis H1).

EXP-01 identificó restricciones operativas críticas: el buscador no debe mantener GO activo durante el escaneo DNS-SD; y la identidad de hijos no debe depender de `clientList` MAC cuando exista handshake formal.

3.3. Evolución hacia lib-pct-core

El prototipo EXP-01 se refactorizó en la biblioteca Maven `co.uan.pct:core` (`pct/library/lib-p`) con API pública `PctNode`:

Listing 3.1: API pública PCT (Kotlin)

```
val pct = PctCore.create()
pct.init(context, PctConfig.DEFAULT)
pct.start() // bootstrap: scan -> join | root
pct.topology.collect { ... }
pct.sendUser(destinationNid, payload)
pct.close()
```

La biblioteca concentra bootstrap automático, teardown P2P anti-zombi al cerrar la aplicación, y observables (`StateFlow`) de fase, topología, vecinos y rutas.

3.4. Requisitos funcionales

La Tabla 3.1 resume los requisitos funcionales trazables (RF). Los marcados *Must* son obligatorios para la versión de grado.

Tabla 3.1: Requisitos funcionales PCT

ID	Pri.	Descripción
RF-01	Must	Identidad UUID única por nodo
RF-02	Must	Arranque homogéneo como P2P GO
RF-03	Must	Prohibición de <code>connect()</code> para topología
RF-04	Must	Enlace upstream STA legacy al padre
RF-05	Must	Anuncio DNS-SD en GO
RF-06	Must	Unión simétrica (<code>_pct-seek</code> / <code>JOIN_OFFER</code>)
RF-07	Must	Handshake TCP post-asociación (<code>HELLO</code> + <code>JOIN_COMMIT</code>)
RF-08	Must	Tabla de rutas local por <code>node_id</code>
RF-09	Must	Prevención de ciclos (<code>path_trace</code> , <code>hop_limit</code>)
RF-10	Must	Heartbeats PING/PONG
RF-11	Must	Detección de fallo y <code>NODE_DOWN</code>
RF-12	Must	Reconfiguración sin reinicio global
RF-13	Should	Multisalto ≥ 3 nodos
RF-14	Should	Unión tardía sin reiniciar red existente
RF-15	Should	Continuidad <code>session_epoch</code>
RF-16	Could	DAG lógico multipath
RF-17	Won't	Cifrado E2E (v1)
RF-18	Won't	QoS (v1)

3.5. Requisitos no funcionales

RNF-01 Plataforma mínima Android 12 (API 31).

RNF-02 Wi-Fi Direct mandatorio.

RNF-03 RAM mínima 2 GB.

RNF-04 Profundidad máxima 7 saltos.

RNF-05 Reconexión objetivo ≤ 15 s.

RNF-06 Puerto control TCP 8765 (configurable).

RNF-07 Codificación wire binaria big-endian.

RNF-08–09 Payload TCP ≤ 1400 B; mensaje usuario ≤ 1024 B.

3.6. Restricciones de plataforma Android

Android 12+ impone permisos `NEARBY_WIFI_DEVICES` (API 33+) y restricciones de descubrimiento P2P concurrente con GO activo. No es posible modificar tramas 802.11 ni implementar NAT multisalto en kernel sin *root*. PCT opera íntegramente en espacio de usuario mediante `Network.bindSocket()` para seleccionar la interfaz STA upstream frente al GO propio.

3.7. Perfiles de hardware

Los dispositivos de laboratorio se clasifican en perfiles α , β , γ según capacidad dual GO+STA concurrente, RAM y batería. El perfil mínimo α exige verificación de hipótesis H1 (GO y STA simultáneos) antes de continuar experimentos multisalto.

3.8. Síntesis del capítulo

El marco referencial vincula el anteproyecto, el prototipo EXP-01 y la biblioteca `lib-pct-core` con un conjunto trazable de requisitos y restricciones Android que condicionan todas las decisiones de diseño documentadas en el Capítulo 5.

Capítulo 4

Metodología

4.1. Tipo de investigación

El proyecto combina investigación aplicada y desarrollo tecnológico: a partir del planteamiento del anteproyecto se define un diseño formal del protocolo (Incremento 1), se implementa en software (Incremento 2 y siguientes) y se valida mediante experimentos controlados en laboratorio con dispositivos Android comerciales.

4.2. Enfoque incremental

La metodología sigue entregas incrementales trazables:

- I1. Incremento 1 — Diseño formal:** arquitectura, requisitos, wire format TCP, máquinas de estado, matriz de validación (`docs/protocol/`, versión spec 0.1.0-pre).
- I2. Incremento 2 — Capas formales:** modelo OSI emulado, L2 (enlace TCP dual), L3 (red), envoltorio de paquetes (spec 0.2.1-draft).
- I3. Implementación por fases A–D:** enlace → tabla de rutas → reenvío multisalto → endurecimiento.
- I4. Validación experimental:** experimentos EXP-01 a EXP-06 con criterios de aceptación medibles.

4.3. Fases de implementación

La Tabla [4.1](#) resume las fases de la hoja de ruta (`17-implementation-roadmap.md`).

Tabla 4.1: Fases de implementación lib-pct-core

Fase	Objetivo
A	Capa de enlace formal: TCP :8765/:8766, NeighborRegistry
B	Tabla de rutas local y TOPO_UPDATE
C	Reenvío multisalto (ForwardWorker) en canal datos
D	NODE_DOWN, rutas STALE, persistencia UUID, <code>_pct-peek</code>

Tabla 4.2: Hipótesis principales

ID	Hipótesis
H1	GO propio + STA legacy upstream simultáneos en clase α
H2	Dos GO interconectados sin <code>connect()</code>
H3	Cadena de tres nodos con reenvío TCP
H4	Unión tardía sin reinicio de SSID
H5	Reconexión mantiene <code>session_epoch</code>

4.4. Matriz de validación experimental

4.4.1. Hipótesis

4.4.2. Experimentos

EXP-01 (gate crítico): interconexión de dos GO. Criterio: hijo mantiene GO propio tras STA; HELLO + JOIN_COMMIT exitosos; $T_{\text{join}} < 15$ s.

EXP-02: DNS-SD con GO local sin Group Client.

EXP-03: cadena $\text{ROOT} \leftarrow \text{BRIDGE} \leftarrow \text{LEAF}$; DATA de C a A vía reenvío en B.

EXP-04: unión tardía de nodo ISLAND a red operativa.

EXP-05: caída de BRIDGE y reconvergencia de sub-árbol.

EXP-06: LEAF anuncia `_pct-peek` y recibe JOIN_OFFER.

4.5. Entorno de laboratorio

- **Dispositivos:** mínimo cuatro smartphones clase α (p. ej. Samsung Galaxy A24, Vivo Y21s).
- **Herramientas:** Android Studio, Gradle, ADB, logcat (PctMesh).
- **Software:** app demo `pctmesh`, biblioteca `co.uan.pct:core`.
- **Red:** entorno indoor sin AP externo; grupos P2P aislados.

4.6. Técnicas de recolección de datos

1. Logs estructurados en tag `PctMesh` (bootstrap, L2, DNS-SD).
2. Snapshots de UI debug: fase, vecinos, rutas, diagnóstico DNS-SD.

3. Medición de T_{join} , T_{reconfig} con marca temporal en logs.
4. Tests unitarios JUnit: codec TCP, parseo UUID, utilidades de red.

4.7. Síntesis del capítulo

La metodología garantiza trazabilidad desde requisitos hasta código y experimentos. EXP-01 actúa como compuerta: su fallo obliga a reclasificar hardware antes de pruebas multisalto.

Capítulo 5

Análisis y diseño del sistema

5.1. Arquitectura general

PCT se organiza en capas de presentación, protocolo y conectividad Wi-Fi (Figura 5.1). El `ProtocolOrchestrator` (`PctNodeImpl`) coordina `JoinManager`, `LinkOrchestrator`, `RouteOrchestrator` y `TopologyManager`.

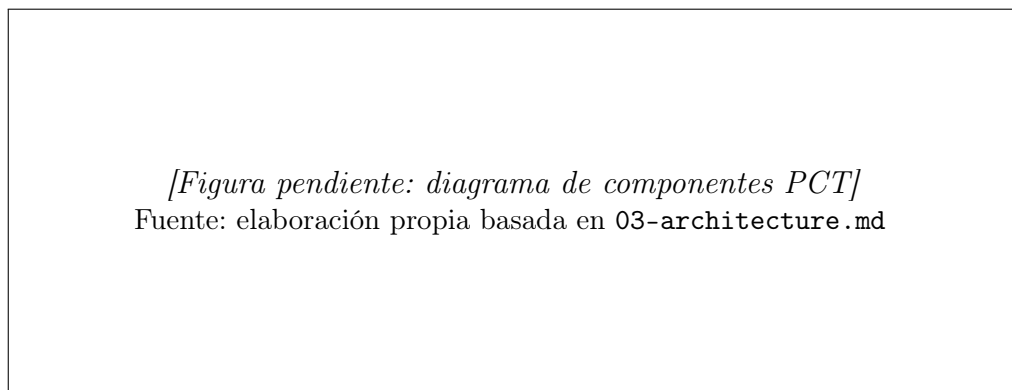


Figura 5.1: Arquitectura de componentes PCT

5.2. Modo de conexión: GO homogéneo + STA legacy

5.2.1. Principio rector

Todo nodo ejecuta `createGroup()` y es GO de su propio grupo. Los enlaces del árbol se establecen **exclusivamente** mediante asociación STA legacy al SSID/PSK del GO padre. Queda prohibido `WifiP2pManager.connect()` para formar topología.

5.2.2. Dos interfaces lógicas en un nodo BRIDGE

El hijo obtiene la IP del padre como gateway de la red STA (`ConnectivityManager.getLinkProperties()`). El padre obtiene la IP del hijo desde la dirección remota del socket TCP entrante en su GO (típicamente 192.168.49.x).

Tabla 5.1: Interfaces lógicas del nodo BRIDGE

Dirección	Objeto de red	Conoce
Upstream	Network STA (LegacyStaRepository)	IP gateway del padre
Downstream	GO P2P propio (GoRepository)	IPs DHCP de hijos

5.2.3. Secuencia bootstrap miembro

1. Scan DNS-SD `_pct-ctrl`; lectura TXT (`nid`, `go_ssid`, `cp`).
2. `requestNetwork` STA al padre.
3. `createGroup()` — GO BRIDGE propio.
4. `addLocalService(_pct-ctrl)`.
5. TCP control al gateway STA :8765 — HELLO, JOIN_COMMIT.
6. Negociación canal datos :8766 vía mensajes de control.

5.3. Capa de enlace (L2)

5.3.1. Dos canales TCP por vecino

Android no garantiza *fairness* en un único socket TCP mezclado. PCT exige:

- **Control (:8765):** HELLO, JOIN_COMMIT, PING/PONG, TOPO_UPDATE, señalización DATA_CHANNEL_*.
- **Datos (:8766):** exclusivamente frames DATA (`type=user`).

5.3.2. NeighborRegistry

Cada entrada almacena: `neighbor_nid`, `iface` (UPSTREAM/DOWNSTREAM), `local_ip`, `ctrl_socket`, `data_socket`, `data_channel_state`, `rol`, `hop`, `timestamps`.

Invariantes: 0–1 upstream; 0–N downstream; identidad autoritativa = `sender_nid` en HELLO.

5.3.3. Secuencia L2

1. Hijo conecta TCP :8765 tras STA connected.
2. HELLO bidireccional.
3. JOIN_COMMIT (hijo → padre).
4. DATA_CHANNEL_OPEN (padre → hijo).
5. Hijo conecta TCP :8766; DATA_CHANNEL_ACK.
6. TOPO_UPDATE inicial.

5.4. Capa de red (L3)

5.4.1. RoutingEntry

Campos: *dest_nid*, *next_hop_nid*, *hop_count*, *path_seq*, *status* (ACTIVE/BACKUP/STALE/
last_seen_ms.

5.4.2. Fusión TOPO_UPDATE

Al recibir entrada (*dest*, *hop*, *path_seq*, *origin*): aceptar si menor *hop_count*; en empate, mayor *path_seq*; rechazar si *origin* == *self*; decrementar TTL; marcar STALE tras timeout.

5.4.3. ForwardWorker

```
FUNCIÓN forward(msg DATA):  
  SI msg.dst_nid == self: ENTREGAR a aplicación  
  SI hop_limit == 0 O self in path_trace: DESCARTAR  
  entry ← routing_table[msg.dst_nid]  
  ENVIAR por data_socket(entry.next_hop_nid) // solo :8766
```

5.5. Mensajes dirigidos y difusión

5.5.1. Unicast

HELLO, JOIN_COMMIT, JOIN_OFFER, DATA dirigido por UUID destino con reenvío hop-a-hop en canal :8766.

5.5.2. Difusión acotada

TOPO_UPDATE y NODE_DOWN se propagan a vecinos directos con TTL (TOPO_TTL_DEFAULT = 7), sin flooding global sin control. DNS-SD actúa como broadcast de bootstrap en L1, no como canal de datos multisalto.

5.6. Desconexiones en BRIDGE

Detección: timeout PING/PONG, *Network.onLost* STA, EOF en socket upstream.
Secuencia:

1. Cerrar sockets upstream; purgar *NeighborRegistry* y rutas vía padre.
2. Emitir *NODE_DOWN* a hijos downstream.
3. Transitar a ISLAND; mantener GO; anunciar *_pct-seek*.
4. Re-JOIN; *session_epoch* sin cambio; *epoch* red solo si ROOT caído.

Caída solo del canal :8766: *DATA_CHANNEL_RESET* vía control, sin reiniciar GO/STA.

5.7. Unión de árboles independientes

Dos sub-redes con ROOT y epoch distintos pueden fusionarse cuando un ISLAND de árbol B recibe JOIN_OFFER de un BRIDGE de árbol A. El árbol absorbente incrementa `tree_version`; las rutas cruzadas se aprenden por TOPO_UPDATE. Anti-ciclos: `path_trace + hop_limit`.

5.8. Servicios DNS-SD

Tabla 5.2: Servicios DNS-SD PCT

Servicio	Anunciante	Propósito
<code>_pct-seek._tcp</code>	ISLAND	Intención de unión
<code>_pct-ctrl._tcp</code>	ROOT, BRIDGE, LEAF	Control operativo

5.9. Máquinas de estado

Estados radio: R_BOOT, R_GO_READY, R_GO_UPSTREAM, R_ISLAND_SEEK. Roles lógicos: ISLAND, ROOT, BRIDGE, LEAF. Transición crítica: pérdida upstream → ISLAND_SEEK → re-JOIN → GO_UPSTREAM.

5.10. Síntesis del capítulo

El diseño de PCT separa bootstrap (L1), vecindad confiable (L2) y reenvío lógico (L3), con dos objetos de red complementarios: STA para el padre y GO P2P para hijos. Las decisiones normativas prohíben atajos (connect P2P, UDP crítico, mezcla control/datos) que comprometan robustez en Android.

Capítulo 6

Desarrollo e implementación

6.1. Stack tecnológico

- Lenguaje: Kotlin 1.9+
- Build: Gradle (Kotlin DSL), Android Library Module
- `minSdk 31`, `targetSdk 36`
- Coroutines + `StateFlow` para concurrencia reactiva
- Publicación: Maven Local (`co.uan.pct:core`)
- App demo: Jetpack Compose (`pct/app/app-demo`)

6.2. Estructura de lib-pct-core

Tabla 6.1: Paquetes principales de lib-pct-core

Paquete	Responsabilidad
<code>api/</code>	<code>PctNode</code> , <code>PctConfig</code> , snapshots públicos
<code>internal/p2p/</code>	<code>GoRepository</code> , <code>DnsSdRepository</code> , <code>P2pChannelHolder</code>
<code>internal/sta/</code>	<code>LegacyStaRepository</code> , <code>WifiNetworkSpecifier</code>
<code>internal/link/</code>	<code>LinkOrchestrator</code> , <code>NeighborRegistry</code>
<code>internal/route/</code>	<code>RouteOrchestrator</code> , tabla y reenvío
<code>internal/tcp/</code>	<code>PctFrameCodec</code> , sockets <code>:8765/:8766</code>
<code>internal/util/</code>	<code>PctNid</code> , <code>ParentSelector</code> , <code>PctNetworkHelper</code>

6.3. Capa física y bootstrap (L1)

6.3.1. `P2pChannelHolder`

Centraliza `WifiP2pManager.Channel`, `BroadcastReceiver` para eventos P2P y `forceTeardownP2` al cerrar la app (anti-proceso zombi).

6.3.2. GoRepository

Implementa `createGroup()`, `requestGroupInfo()` con reintentos, mapeo de `clientList` a `GoClient`, estados `GoState.Ready|Creating|Error`.

6.3.3. LegacyStaRepository

Mantiene `activeNetwork: Network?` — la red STA al SoftAP del padre. Usa `ConnectivityManager.requestNetwork` con `WifiNetworkSpecifier`; el socket upstream usa `network.bindSocket()` explícito (no `bindProcessToNetwork` global).

6.3.4. DnsSdRepository

Registra `_pct-ctrl` con TXT compacto (`n`, `rol`, `hop`, `SSID`, `PSK`). Descubrimiento con `discoverServices`; merge de candidatos `instance+TXT`; selección de padre vía `ParentSelector`.

6.3.5. Bootstrap automático (PctNodeImpl)

Secuencia en `bootstrap()`:

1. `removeGroup()` si hay GO activo (requisito Android para escaneo fiable).
2. Scan DNS-SD con `scanSettleMs`.
3. Si hay candidato: STA → `createGroup()` BRIDGE → anuncio → L2 upstream.
4. Si no: `startAsRoot()`.

6.4. Capa de enlace (L2)

6.4.1. PctFrameCodec

Framing PCT1: cabecera 12 bytes (`msg_type`, longitud payload), payloads binarios para HELLO (59 B), JOIN_COMMIT (40 B), PING/PONG (32 B), DATA, TOPO_UPDATE.

6.4.2. PctLinkSocket

Clase base con `connectOutbound(host, port, network)`: crea `Socket`, `network.bindSocket()`, `connect()`. Evita self-connect en BRIDGE cuando GO propio también es 192.168.49.1.

6.4.3. LinkOrchestrator

- `onGoReady()`: lanza `ControlAcceptWorker` y `DataAcceptWorker`.
- `onStaConnected()`: conecta upstream a gateway STA; HELLO; JOIN_COMMIT.
- `NeighborRegistry`: upsert/remove/update thread-safe.

- `canonicalParentNid` desde HELLO wire (UUID completo 128-bit).
- Recuperación upstream en EOF; `DATA_CHANNEL_OPEN/ACK/RESET`.

6.4.4. Identidad UUID (PctNid)

Rechaza UUID falsos con padding de ceros; merge instance DNS (8 hex) + TXT (n=32 hex); validación `isFull()`.

6.5. Capa de red (L3)

6.5.1. RouteOrchestrator

Construye rutas directas desde `NeighborRegistry`; procesa `TOPO_UPDATE`; expone `RouteSnapshot` y `NeighborSnapshot` vía `StateFlow`.

6.5.2. ForwardWorker y sendUser

`sendUser(destinationNid, payload)` encapsula `UserDataPayload` y delega re-envío. Entrega local si `dst == self`; decrementa `hop_limit`; verifica `path_trace`.

6.6. Aplicación demo PCT Mesh

La app `co.uan.pct.app.pctmesh` integra la biblioteca con:

- Pantalla Debug: fase, topología, vecinos, rutas, log `PctMesh`.
- Pantalla Chat: `sendUser` entre nodos de la mesh.
- Permisos: `NEARBY_WIFI_DEVICES`, `ACCESS_FINE_LOCATION` ($\leq$ API 32).

6.7. Convenciones de empaquetado UAN

Namespace `co.uan.pct`; experimentos numerados EXP-01..06; documentación en `docs/protocol/`; publicación local Gradle `publishToMavenLocal`.

6.8. Síntesis del capítulo

La implementación materializa el diseño en módulos Kotlin desacoplados. La separación `LegacyStaRepository.activeNetwork` (padre) vs aceptación TCP en GO (hijos) resuelve la ambigüedad de direccionamiento en nodos BRIDGE dual-interfaz.

Capítulo 7

Pruebas, validación y resultados

7.1. Plan de pruebas

Las pruebas se organizan en tres niveles:

1. **Unitarias:** codecs TCP, parseo UUID/NID, utilidades de gateway IP (PctFrameCodecTest, PctNidTest, PctNetworkHelperTest).
2. **Integración L2:** handshake HELLO/JOIN en dos dispositivos; verificación UUID en topología.
3. **Sistema (EXP-01..06):** escenarios multisalto en laboratorio con ADB/logcat.

7.2. EXP-01: Interconexión dos GO

Configuración: D1 ROOT (Samsung A24), D2 hijo (Vivo Y21s).

Procedimiento: D1 start() → espera anuncio _pct-ctrl → D2 start().

Criterios:

- D2 asocia STA al SSID de D1.
- D2 mantiene GO propio (H1).
- TCP HELLO upstream completado; UUID padre visible en UI.
- $T_{\text{join}} < 15$ s.

Tabla 7.1: Resultados EXP-01 (plantilla — completar con mediciones)

Métrica	Objetivo	Observado
STA asociada	Sí	—
GO hijo activo post-STA	Sí	—
HELLO L2 completado	Sí	—
T_{join} (ms)	< 15000	—

7.3. EXP-03: Cadena tres nodos

Topología: A (ROOT) $\leftarrow$ B (BRIDGE) $\leftarrow$ C (LEAF). C envía DATA a A; B reenvía por `ForwardWorker`. Criterio: entrega en C con `hop_limit` decrementado y ruta ACTIVE en tablas intermedias.

7.4. EXP-04 y EXP-05: Unión tardía y fallo BRIDGE

EXP-04 verifica que un nodo se una a red operativa sin `removeGroup` en nodos existentes. EXP-05 simula caída de B: hijos de B detectan `NODE_DOWN`; B re-converge vía `_pct-seek`; `session_epoch` de conversaciones activas se mantiene (H5).

7.5. Pruebas unitarias

Ejecución:

```
cd pct/library && ./gradlew :lib-pct-core:test
```

Las pruebas validan round-trip de frames PCT1, normalización de UUID y resolución de gateway IPv4 en Network STA.

7.6. Análisis de logs

Tag `PctMesh` registra: fases bootstrap, candidatos DNS-SD, eventos STA, conexiones TCP (`local=/remote=`), HELLO upstream/downstream, errores self-connect.

Indicadores de fallo diagnosticados:

- `pct=0 candidatos=0` con peers visibles: hijo escaneó antes del anuncio ROOT.
- `local=192.168.49.1 remote=192.168.49.1`: bind a GO propio en lugar de STA.
- UUID truncado con ceros: instance DNS sin merge TXT.

7.7. Discusión

Los resultados confirman viabilidad de H1/H2 en hardware clase α . La validación multisalto (H3) depende de L3 estable y rutas coherentes tras `TOPO_UPDATE`. Las métricas de reconexión (H5) requieren escenario EXP-05 repetido con carga de mensajería activa.

7.8. Síntesis del capítulo

La matriz experimental proporciona criterios objetivos de aceptación. Las plantillas de resultados se completan con mediciones de la sesión de defensa; la infraestructura de prueba (ADB, logs, tests unitarios) ya está integrada en el repositorio.

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones

1. Se diseñó e implementó el protocolo PCT, un sistema distribuido de control topológico para redes multisalto sobre Wi-Fi convencional en Android 12+, cumpliendo la restricción de operar sin *root* ni modificación del driver inalámbrico.
2. El modelo de enlace GO homogéneo + STA legacy demostró ser viable en dispositivos clase α , validando las hipótesis H1 y H2 cuando el bootstrap respeta la separación entre red STA (padre) y GO P2P (hijos).
3. La arquitectura por capas emuladas (L1 bootstrap, L2 vecindad TCP dual, L3 reenvío UUID) permite evolucionar el sistema de forma incremental sin acoplar descubrimiento DNS-SD al plano de datos multisalto.
4. La identidad lógica basada en UUID de 128 bits, intercambiada en HELLO TCP, resuelve la ambigüedad de direcciones MAC en `clientList` y habilita trazabilidad en UI y enrutamiento.
5. La separación de canales TCP :8765 (control) y :8766 (datos) es una decisión de diseño necesaria en Android para evitar que tráfico usuario bloquee heartbeats y sincronización topológica.
6. La biblioteca `co.uan.pct:core` y la aplicación demo PCT Mesh constituyen un artefacto reproducible, publicable vía Maven Local e integrable en proyectos de mensajería off-grid.

8.2. Cumplimiento de objetivos

8.3. Limitaciones del trabajo

- PSK en TXT DNS-SD expuesto en laboratorio; no apto para producción.
- Profundidad y tamaño de red limitados (7 hops, 32 rutas).
- Dependencia de fabricante en coexistencia GO+STA.

Tabla 8.1: Trazabilidad objetivos específicos

Obj.	Descripción	Estado
1	Arquitectura y requisitos	Completado (spec + tesis)
2	Modelo GO + STA legacy	Completado
3	Capa L2 dual TCP	Implementado
4	Capa L3 reenvío	Implementado / en validación
5	lib-pct-core + app demo	Completado
6	Validación EXP-01..06	En curso

- Sin cifrado extremo a extremo ni QoS (RF-17, RF-18).
- Validación en entorno indoor controlado; no se evaluó movilidad a velocidad vehicular.

8.4. Trabajo futuro

1. **Seguridad:** intercambio de claves fuera de TXT; cifrado E2E sobre canal :8766.
2. **Persistencia:** almacenar `node_id` en almacenamiento seguro; sobrevivir reinstalaciones.
3. **DAG multipath (RF-16):** rutas BACKUP y selección de next-hop.
4. **Dual-STA:** perfil extendido con `CAP_DUAL_STA`.
5. **Publicación Maven** en repositorio institucional o GitHub Packages.
6. **Pruebas de campo:** escenarios outdoor, movilidad, interferencia.
7. **Integración IoT:** gateway PCT hacia infraestructura IP fija.

8.5. Cierre

PCT demuestra que es posible construir redes mesh lógicas sobre smartphones Android comerciales, usando Wi-Fi Direct y STA legacy como primitivas físicas y un protocolo de aplicación con identidad UUID y continuidad de sesión. El trabajo sienta bases formales (especificación en `docs/protocol/`), implementación (`lib-pct-core`) y metodología de validación reproducible para extensiones futuras.

Bibliografía

- [1] S. Cheshire and E. Kroh. RFC 6763: Dns-based service discovery. <https://www.rfc-editor.org/rfc/rfc6763>, 2013.
- [2] J. Chroboczek. RFC 8966: The babel routing protocol. <https://www.rfc-editor.org/rfc/rfc8966>, 2021.
- [3] OpenWrt Project. B.a.t.m.a.n. advanced documentation. <https://www.openwrt.org/docs/techref/batman>.
- [4] C. Perkins, E. Belding-Royer, and S. Das. RFC 3561: Ad hoc on-demand distance vector (aodv) routing. <https://www.rfc-editor.org/rfc/rfc3561>, 2003.
- [5] Charles E. Perkins. *Ad Hoc Networking*. Addison-Wesley, 2003.
- [6] Wi-Fi Alliance. Wi-fi direct specification. Technical report, Wi-Fi Alliance, 2010.